# K8S-02: DynaKube Operator Deployment

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 2 of 13 | **Created:** January 2026 | **Last Updated:** 07/30/2026

## Installing and Configuring the Dynatrace Operator

The DynaKube operator is the recommended way to deploy Dynatrace monitoring in Kubernetes. This notebook covers installation via Helm, configuration options, and deployment modes for different use cases.

---

## Table of Contents



---

## Prerequisites

| Requirement | Details |
|---|---|
| **Dynatrace Environment** | SaaS tenant with API tokens |
| **Kubernetes Cluster** | v1.24+ with admin access |
| **Helm** | v3.x installed |
| **kubectl** | Configured for your cluster |
| **API Tokens** | Operator token (Platform Token recommended) + Data ingest token |

> **Operator token type:** new automation should use **Platform Tokens** ( `dt0s16` ) with `Authorization: Bearer` for the Operator token — note that **the Operator itself accepts platform tokens only from Operator 1.10.0** (released July 15, 2026): *"No immediate action is required and existing access tokens continue to be accepted."* On clusters running an earlier Operator, keep the classic access (API) token — it remains the working path. The OneAgent installer download path still uses Classic API Tokens ( `dt0c01` with `Authorization: Api-Token` ); this is expected and supported. See the IAM series for parameterized policy patterns and the ONBRD-99 Recommended Defaults card for the canonical token / configuration / extension stack.
>
> **Version Support Policy:** OneAgent and ActiveGate versions are supported for **9 months (Standard)** or **12 months (Enterprise)**. Third-party technologies are supported for 6 months beyond vendor EOL. See Support Policy (Dynatrace).

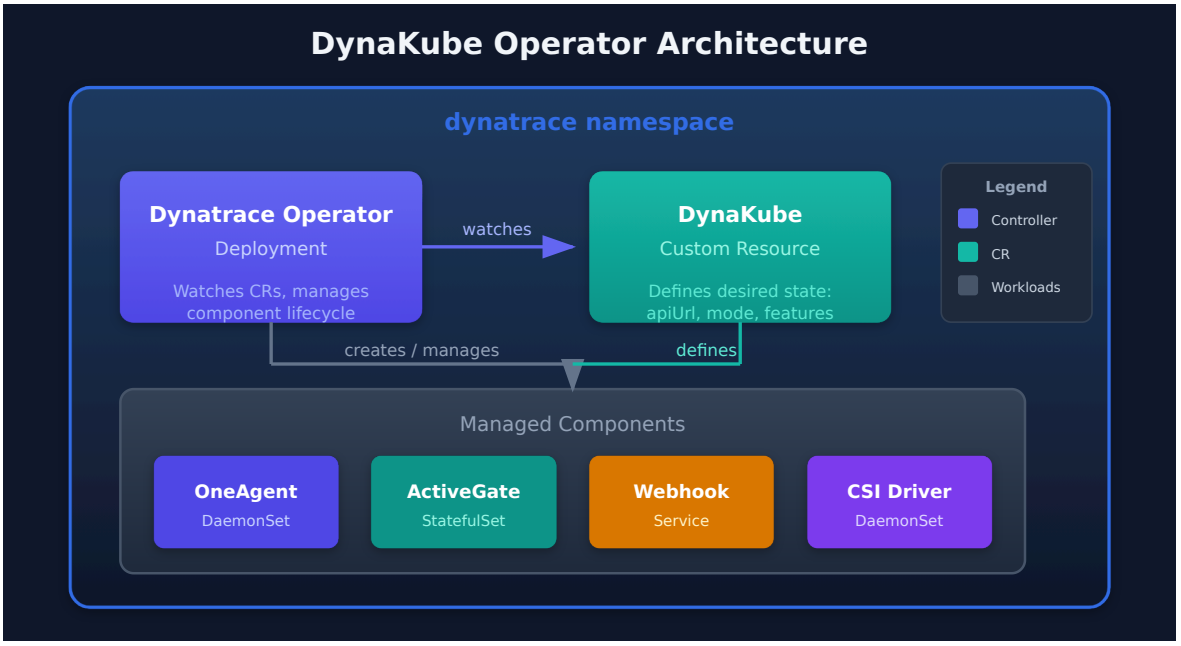
## 1. Operator Overview

The Dynatrace Operator manages the complete lifecycle of Dynatrace monitoring components.

### What the Operator Manages

| Component | Description | Managed By |
|---|---|---|
| **OneAgent DaemonSet** | Node-level monitoring | Operator |
| **ActiveGate StatefulSet** | Routing and K8s API access | Operator |
| **Webhook** | Code module injection | Operator |
| **CSI Driver** | Volume-based code modules | Operator |

### Operator Architecture




## 2. Prerequisites Setup

### Required API Tokens

Create two tokens in Dynatrace with these scopes:

**Operator Token:**

| Scope | Purpose |
|---|---|
| `activeGateTokenManagement.create` | ActiveGate tokens |

| Scope | Purpose |
|---|---|
| `entities.read` | Entity information |
| `settings.read` | Read settings |
| `settings.write` | Write settings |
| `DataExport` | Export data (optional) |
| `InstallerDownload` | Download OneAgent |

**Data Ingest Token:**

| Scope | Purpose |
|---|---|
| `metrics.ingest` | Ingest metrics |
| `logs.ingest` | Ingest logs |
| `events.ingest` | Ingest events (optional) |

### Create Namespace and Secret

```
# Create namespace
kubectl create namespace dynatrace

# Create secret with tokens
kubectl create secret generic dynakube \
  --namespace dynatrace \
  --from-literal=apiToken=&lt;OPERATOR_TOKEN&gt; \
  --from-literal=dataIngestToken=&lt;DATA_INGEST_TOKEN&gt;
```

## 3. Helm Chart Installation

### Chart Reference: OCI (documented) or Classic Repository

Dynatrace documents the operator chart as an **OCI reference** on a public registry:

```
# Documented form — OCI registry, version pinned explicitly
helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --version 1.10.1 \
  --create-namespace \
  --namespace dynatrace \
  --install \
  --atomic
```

`--install` makes the same command serve a first install and an upgrade; `--atomic` rolls the release back automatically if any component fails to start.

The classic `helm repo add` index is still served and still works, so automation built on it is **stale, not broken** — there is no urgency to rewrite it, and it is not the long-deprecated `dynatrace/helm-charts` repository. Prefer the OCI form for new automation; the classic form remains useful for `helm search repo` version discovery:

```
# Classic form — still functional
helm repo add dynatrace https://raw.githubusercontent.com/Dynatrace/dynatrace-
operator/main/config/helm/repos/stable
helm repo update

# View available versions
helm search repo dynatrace/dynatrace-operator --versions

# Install operator only (DynaKube CR applied separately)
helm install dynatrace-operator dynatrace/dynatrace-operator \
  --namespace dynatrace \
  --create-namespace \
  --wait
```

> **Always pass `--version`.** An unpinned install silently takes whatever is newest at run time, which is how a cluster ends up on a release its own notes tell you to skip. §8 covers how to choose the version.

## What the Install Creates

| Component | Kind | Role |
|---|---|---|
| `dynatrace-operator` | Deployment | Reconciles DynaKube CRs |

| Component | Kind | Role |
|---|---|---|
| `dynatrace-webhook` | Deployment | Admission webhook — performs code-module injection |
| `dynatrace-oneagent-csi-driver` | DaemonSet | Serves OneAgent code modules to pods as volumes (when `csidriver.enabled: true`) |
| `webhook-cert-generator` | init container | Creates the `dynatrace-webhook-certs` secret the webhook needs before it can start (Operator 1.10.0+) |
| `crd-storage-migrator` | init container | Migrates stored DynaKube objects to the current CRD storage version (Operator 1.10.0+) |

Naming the two init containers matters during triage: an operator pod stuck at `Init:0/1` is waiting on one of them, and

```
kubectl -n dynatrace get pod -l app.kubernetes.io/name=dynatrace-operator \
  -o jsonpath='{.items[*].spec.initContainers[*].name}'
```

tells you which is present. K8S-09 §2 maps each to its failure mode.

## Install with Custom Values

```
# Create values file
cat &gt; dt-values.yaml &lt;&lt; 'EOF'
# Operator configuration
operator:
  image: ""             # Override only for a private mirror; empty = public
registry
  customPullSecret: ""  # Required only when pulling from a private registry

# Platform-specific settings
platform: "kubernetes"  # or "openshift"

# Webhook configuration
webhook:
  hostNetwork: false

# CSI driver
csidriver:
  enabled: true
EOF

# Install with values
helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --version 1.10.1 \
  --namespace dynatrace \
  --create-namespace \
  --values dt-values.yaml \
  --install \
  --atomic
```

> **Private mirror vs. public-registry auto-update (Operator 1.10.0+).** Leaving `operator.image` and `customPullSecret` empty pulls from the public registry, which is what enables component **auto-update with multi-architecture image support** — **ARM64, x86-64, s390x, and PPC64le** — so a mixed-architecture node pool resolves the right image per node with no per-arch configuration. Mirroring into a private registry is fully supported, but you then own re-mirroring every architecture you run, and auto-update can only pick up what you have mirrored. Decide this before a mixed-arch rollout, not during one. Operator 1.10.0 was released July 15, 2026 and estates adopt it on their own schedule — on earlier Operators the pre-1.10 pull behavior remains the working path, unchanged.

# 4. DynaKube Custom Resource

The DynaKube CR defines how monitoring is deployed.

## Minimal DynaKube (cloudNativeFullStack)

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  # Your Dynatrace environment URL
  apiUrl: https://ENVIRONMENT_ID.live.dynatrace.com/api

  # OneAgent deployment mode
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/master
          operator: Exists
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists

  # ActiveGate for Kubernetes monitoring
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
      - dynatrace-api
```

> **API Version Note.** `v1beta6` is the current DynaKube API version (introduced in Operator 1.8.0) and is what **new DynaKubes should use** — the production CR in K8S-14 is written against it. `v1beta5` remains accepted, so **existing manifests need no emergency rewrite**; the minimal example above is deliberately left on `v1beta5` to demonstrate that an older-version manifest still applies cleanly.
>
> The deprecation train is what makes this load-bearing at upgrade time:

| API version | Status |
| --- | --- |
| `v1beta6` | Current — use for new DynaKubes |
| `v1beta5` | Accepted — no rewrite required |
| `v1beta4` | **Deprecated** as of Operator 1.10.x — migrate at your convenience, before it follows `v1beta3` |
| `v1beta3` | **Removed in Operator 1.9.0** — an apply against it fails outright on 1.9.0 and later |

Audit before crossing a version that removes one. Check what the cluster currently serves:

`bash > kubectl get dynakube -A -o jsonpath='{.items[*].apiVersion}' >`

That reports the version the API server serves, which is not necessarily the string in your committed YAML — so also grep the manifests your GitOps controller applies ( `grep -r "apiVersion: dynatrace.com/" .` ). A manifest pinned to a removed version fails at apply time, which in a GitOps setup surfaces as a sync error rather than an obvious upgrade failure. §8 folds this into the pre-upgrade checklist.

### Apply the DynaKube

```
# Apply DynaKube CR
kubectl apply -f dynakube.yaml

# Watch deployment progress
kubectl -n dynatrace get dynakube -w
```

## 5. Deployment Modes Explained

### cloudNativeFullStack (Recommended)

```
oneAgent:
  cloudNativeFullStack:
    # Injected via webhook using CSI driver volumes
```

| Pros | Cons |
|---|---|
| No privileged containers for apps | Requires CSI driver |
| Best for multi-tenant clusters | Slightly more complex |
| Independent app/infra monitoring | |

## classicFullStack (Legacy — Migrate Existing Clusters, Not Just New Ones)

```
oneAgent:
  classicFullStack:
    # OneAgent DaemonSet provides code modules via host mounts
```

| Pros | Cons |
|---|---|
| Simpler architecture | All pods share OneAgent |
| Single component | Requires hostPath mounts |
| | **Not a ready state for the latest Dynatrace** — see below |

> **"Avoid for new deployments" understates this.** The ready-made *Check your upgrade readiness* dashboard treats a DynaKube using `classicFullStack` as **not ready** for the latest Dynatrace — the check is on the cluster's current state, not on when it was created. A cluster that has been running `classicFullStack` happily for two years is flagged exactly like a new one.
>
> So this is migration work with a deadline attached, not a style preference. Move existing clusters to `cloudNativeFullStack` (or `applicationMonitoring` / `hostMonitoring` where those fit) as part of upgrade preparation rather than waiting for a rebuild. The readiness check also covers Operator ≥ 1.10, the connecting ActiveGate ≥ 1.343, and the image source — see §3 and the Classic → Gen3 doorway in the `-START-HERE-` playbook.

### applicationMonitoring

```
oneAgent:
  applicationMonitoring:
    # Only application-level monitoring, no infrastructure
    useCSIDriver: true
```

| Pros | Cons |
| --- | --- |
| Minimal footprint | No infrastructure visibility |
| No node-level access needed | No container metrics |
| Good for managed K8s | |

### hostMonitoring

```
oneAgent:
  hostMonitoring: {}
```

| Pros | Cons |
| --- | --- |
| Infrastructure only | No application tracing |
| Lightweight | No code-level insight |

## 6. Configuration Options

### Resource Limits

```
spec:
  oneAgent:
    cloudNativeFullStack:
      resources:
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 300m
          memory: 512Mi
```

## Node Selectors and Tolerations

```
spec:
  oneAgent:
    cloudNativeFullStack:
      nodeSelector:
        kubernetes.io/os: linux
      tolerations:
        - key: "dedicated"
          operator: "Equal"
          value: "monitoring"
          effect: "NoSchedule"
```

## Namespace Selectors

Control which namespaces get injection:

```
spec:
  namespaceSelector:
    matchLabels:
      dynatrace-injection: enabled
```

Or exclude specific namespaces:

```
spec:
  namespaceSelector:
    matchExpressions:
      - key: dynatrace-injection
        operator: NotIn
        values:
          - disabled
```

## Custom ActiveGate Configuration

```
spec:
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
    replicas: 2
    resources:
      requests:
        cpu: 500m
        memory: 512Mi
      limits:
        cpu: 1000m
        memory: 1Gi
    topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: ScheduleAnyway
```

## Feature Flags and Annotations

Control injection at the pod level:

| Annotation | Effect |
|---|---|
| `oneagent.dynatrace.com/inject: "false"` | Disable injection for pod |
| `oneagent.dynatrace.com/inject: "true"` | Force enable injection |
| `oneagent.dynatrace.com/technologies: "java,nodejs"` | Limit technologies |

DynaKube feature flags:

```
spec:
  oneAgent:
    cloudNativeFullStack:
      # Note: ONEAGENT_ENABLE_VOLUME_STORAGE defaults to true when CSI driver
      # is enabled — no need to set explicitly
```

## Templates Configuration

Configure OTel Collector and Log Monitoring via `spec.templates`:

```
spec:
  templates:
    otelCollector:
      # ⚠️ Always pin to specific version - avoid 'latest'.
      # Current latest as of May 2026 is v0.48.0; verify against the live
releases:
      # https://github.com/Dynatrace/dynatrace-otel-collector/releases
      imageRef:
        repository: public.ecr.aws/dynatrace/dynatrace-otel-collector
        tag: "0.48.0"
      resources:
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 500m
          memory: 512Mi

    logMonitoring:
      resources:
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 500m
          memory: 512Mi
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists
```

> **Note:** Log monitoring is enabled with `spec.logMonitoring: {}` but resource configuration goes under `spec.templates.logMonitoring`.

## OTel Collector and Log Monitoring Sizing Guide

| Environment | CPU Limit | Memory Limit | Notes |
|---|---|---|---|
| Dev/Test | 200m | 256Mi | Low telemetry volume |
| Staging | 500m | 512Mi | Medium volume |
| Production | 1000m | 1Gi | High volume |

# 7. Verification and Validation

## Check Operator Status

```
# Operator pods
kubectl -n dynatrace get pods -l app.kubernetes.io/name=dynatrace-operator

# DynaKube status
kubectl -n dynatrace get dynakube

# Detailed status
kubectl -n dynatrace describe dynakube dynakube
```

## Check OneAgent Deployment

```
# OneAgent DaemonSet
kubectl -n dynatrace get daemonset -l app.kubernetes.io/component=oneagent

# OneAgent pods on each node
kubectl -n dynatrace get pods -l app.kubernetes.io/component=oneagent -o wide
```

## Check ActiveGate

```
# ActiveGate StatefulSet
kubectl -n dynatrace get statefulset -l app.kubernetes.io/component=activegate

# ActiveGate pods
kubectl -n dynatrace get pods -l app.kubernetes.io/component=activegate
```

## DynaKube Status Phases

| Phase | Meaning |
|---|---|
| `Running` | All components healthy |
| `Deploying` | Components being created |
| `Error` | Check events for details |

```
// Verify Kubernetes cluster is reporting to Dynatrace
fetch dt.entity.kubernetes_cluster
| fields entity.name, tags
| sort entity.name asc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_CLUSTER"
//   | fields name, tags
//   | sort name asc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration inventory keep the classic
query above.
// Note: entity tags are not a flat "tags" field on Smartscape (resolve via
getNodeField).
```

```
// Check OneAgent deployment health via events
fetch logs, from:-1h
| filter matchesPhrase(content, "dynatrace") and matchesPhrase(content,
"oneagent")
| fields timestamp, content
| sort timestamp desc
| limit 20
```

# 8. Upgrading the Operator

## Pre-Upgrade Checks

An operator upgrade is not only a version bump — it can change **chart defaults** and it can **remove a DynaKube API version**. Both are silent in a values diff. Run these three checks before every upgrade.

**1. Will a changed chart default alter my deployment?**

`--reuse-values` preserves *the values you set*. It does **not** shield you from a changed default for a value you never set, because there is nothing of yours to reuse. Render the incoming chart and diff it against what is live:

```
# Render the target version with your current values
helm get values dynatrace-operator -n dynatrace &gt; current-values.yaml

helm template dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --version 1.10.1 \
  --namespace dynatrace \
  --values current-values.yaml &gt; new.yaml

# Compare against the rendered manifests currently deployed
helm get manifest dynatrace-operator -n dynatrace &gt; live.yaml
diff live.yaml new.yaml
```

The concrete case this catches: **Operator 1.10.0 flipped the ActiveGate `TopologySpreadConstraint` default from `DoNotSchedule` to `ScheduleAnyway`.** Expect an **ActiveGate restart on upgrade** — the pod spec changes, so the StatefulSet rolls. Nothing is broken; plan for the brief gap in ActiveGate-served capabilities (Kubernetes API monitoring, routing, metrics ingest) rather than being surprised by it.

If your DynaKube sets `whenUnsatisfiable` explicitly, **your value wins** and the default flip does not apply to you. The ActiveGate example in §6 already sets `ScheduleAnyway` explicitly, which is the recommended posture: an explicit constraint makes the behavior independent of chart defaults across upgrades.

**2. Does my DynaKube declare an API version the target Operator still serves?**

```
# What the cluster serves today
kubectl get dynakube -A -o jsonpath='{.items[*].apiVersion}'

# What your committed manifests declare
grep -r "apiVersion: dynatrace.com/" .
```

`v1beta3` was **removed in Operator 1.9.0** — after that upgrade an apply against it fails outright. `v1beta4` is **deprecated in 1.10.x**, so treat it as the next one to migrate off. Target `v1beta6` for anything you are rewriting anyway; `v1beta5` remains accepted. Full table in §4.

**3. Will the new injected pod spec still be admitted?**

On OpenShift, and on any cluster with restrictive admission policy, an operator upgrade changes injected pod specs — and those specs are re-validated on admission. **FAQ-13 §6 ("Assessing Impact Before You Upgrade")** generalizes this into a pre-upgrade assessment checklist; run it before an OpenShift upgrade rather than after.

## Helm Upgrade Process

```
# Check current version
helm list -n dynatrace

# Upgrade to a specific, validated version (OCI — documented form)
helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --version 1.10.1 \
  --namespace dynatrace \
  --reuse-values \
  --atomic \
  --wait

# Classic-repo equivalent (still functional)
helm repo update
helm upgrade dynatrace-operator dynatrace/dynatrace-operator \
  --version 1.10.1 \
  --namespace dynatrace \
  --reuse-values \
  --wait
```

> **Pin the target, do not upgrade to "latest".** Omitting `--version` takes whatever is newest at run time — which is how a cluster lands on a release its own notes advise skipping.

## Version Compatibility

| Operator Version | Min K8s | Max K8s | Notes |
|---|---|---|---|
| 1.7.x | 1.23 | 1.29 | Supported (verify against your environment) |
| 1.8.x | 1.24 | 1.30 | Supported |
| 1.9.x | 1.25 | 1.31 | Supported |

| Operator Version | Min K8s | Max K8s | Notes |
|---|---|---|---|
| 1.10.x | — | — | **Current line. Recommended: v1.10.1** (July 2026). **Skip 1.10.0** — its own release notes advise skipping it (auto-update defect) and it is now flagged `prerelease: true` on the GitHub releases page. **v1.10.2** was published 07/30/2026 and is the newest tag, but its release-notes page is not yet available and the GitHub release body carries no changelog, so there is no citable statement of what it changes — confirm its contents before adopting it. Dynatrace does not publish a min/max Kubernetes range for this line; verify in the release notes for your specific tag |

> **Operator support window:** the Standard 9-month / Enterprise 12-month support policy means versions older than v1.7 are likely past EOL. Plan upgrades accordingly. Older versions may still function but will not receive security or compatibility fixes.

### Rollback if Needed

```
# View history
helm history dynatrace-operator -n dynatrace

# Rollback to previous version
helm rollback dynatrace-operator 1 -n dynatrace
```

## Next Steps

Now that the operator is deployed, proceed to:

| Next Notebook | Topic |
|---|---|
| **K8S-03: GitOps for DynaKube** | Manage DynaKube with ArgoCD/Flux |
| **K8S-04: Cluster Health Monitoring** | Deep-dive into cluster metrics |
| **K8S-05: Workload Monitoring** | Application-level observability |

## Summary

In this notebook, you learned:

- Dynatrace Operator architecture and components
- Prerequisites: API tokens and namespace setup
- Helm chart installation with custom values
- DynaKube CR structure and configuration
- Deployment modes and when to use each
- Configuration options for resources, selectors, and features
- Verification commands and status phases
- Upgrade and rollback procedures

---

## References

- Setup on Kubernetes — deployment (DT docs) — application-observability, full-stack-observability, platform-observability deployment paths
- DynaKube parameters reference (DT docs) — full CRD spec (renamed from `dynakube-crd`)
- DynaKube feature flags (DT docs) — reference list of supported feature-flag annotations
- Helm chart values.yaml (Dynatrace GitHub) — every Helm install option
- Dynatrace Operator releases (Dynatrace GitHub) — recommended pin is **v1.10.1** (July 2026). Skip 1.10.0 (auto-update defect; now flagged `prerelease: true`). **v1.10.2** is the newest tag as of 07/30/2026 but ships without a published changelog — verify its contents before adopting. Check before each install

---